

Codage du texte

Comment le texte est-il représenté dans un ordinateur ?

A	é	愛	😎	🦴
$(41)_{16}$	$(C3A9)_{16}$	$(E6849B)_{16}$	$(F09F988E)_{16}$	$(F0938080)_{16}$

Nicholas Wolff · CC BY-SA 4.0

Sommaire

Différents codages de texte en nombres

Codage ne veut pas dire chiffrement.

1. **Codages du texte**
   ASCII, Character Sets, Unicode avec UTF-8
2. **Exemples UTF-8**
   Coder et décoder un caractère
3. **Malentendus**
   Sur les conventions
4. **Propriétés d'UTF-8**
   Avantages et inconvénients

Un codage par signal électrique, antérieur à l'informatique

Activité · 3 min

Connaissez-vous déjà des manières de coder du texte pour le transmettre à travers un câble électrique ?

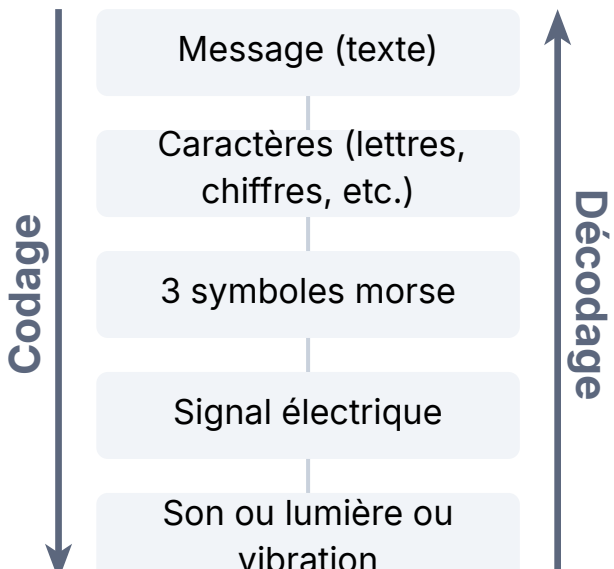
morse.mp3

Exemple : le morse

Code morse international

1. Un point dure une unité.
2. Un trait dure trois unités.
3. Entre les éléments d'une même lettre : une unité.
4. Entre deux lettres : trois unités.
5. Entre deux mots : sept unités.

A	·−	N	−·	0	−−−−−
B	−···	O	−−−	1	·−−−−
C	−·−·	P	·−−·	2	··−−−
D	−··	Q	−−·−	3	···−−
E	·	R	·−·	4	····−
F	··−·	S	···	5	·····
G	−−·	T	−	6	−····
H	····	U	··−	7	−−···
I	··	V	···−	8	−−−··
J	·−−−	W	·−−	9	−−−−·
K	−·−	X	−··−		
L	·−··	Y	−·−−		
M	−−	Z	−−··		



Le texte dans un ordinateur et entre ordinateurs

- Dans un ordinateur, tout est représenté en binaire (avec des 0 et des 1).
- Nous savons déjà comment représenter les nombres en binaire.
- Pour représenter les lettres d'un texte, il suffit de trouver une **convention** pour faire correspondre chaque lettre à un nombre.

ASCII

American **S**tandard **C**ode for **I**nformation **I**nterchange – Standard créé en 1963

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	*SP*	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Pour **coder** un caractère :

- On trouve le premier symbole hexadécimal à gauche de la ligne contenant la lettre.
- On trouve le deuxième symbole hexadécimal en haut de la colonne contenant la lettre.

Par exemple pour coder K :

K est sur la ligne **4_**, colonne **B**.

Donc K se code **4B** en hexadécimal.

ASCII : exercices

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	*SP*	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Activité · 8 min

1. Coder les lettres de votre prénom en hexadécimal à l'aide de la table.
2. Décoder ce texte binaire (convertir vers l'hexadécimal en utilisant la table de droite, puis de l'hexadécimal au texte en utilisant la table ASCII) : `01010011 01001111 01010011`
3. Combien de bits sont utilisés par caractère ?
4. Peut-on coder la phrase « Un âne est-il passé par là ? » à l'aide de cette table (justifier la réponse) ?

ASCII : exercices

1. Coder les lettres de votre prénom en hexadécimal à l'aide de la table.

On utilise la table ASCII pour trouver le nombre hexadécimal correspondant à chaque lettre. Par exemple, « Alan » devient la séquence de 4 octets : `41 6C 61 6E`

2. Décoder ce texte binaire.

- On commence par convertir chaque octet en hexadécimal.
- Puis on cherche chaque caractère dans la table ASCII.

Binaire	`0101'0011`	`0100'1111`	`0101'0011`
Hexadécimal	`5 3`	`4 F`	`5 3`
Caractères	**S**	**O**	**S**

3. Combien de bits sont utilisés par caractère ?

Le plus grand caractère de la table ASCII apparaît en bas à droite. Son numéro est $(7F)_{16}$, ce qui vaut $(0111'1111)_2$, ce qui est équivalent à $(111'1111)_2$, un nombre binaire sur 7 bits.

Donc : en ASCII, **7 bits** sont utilisés par caractère.

4. Peut-on coder la phrase « Un âne est-il passé par là ? » à l'aide de cette table (justifier la réponse) ?

Non, car on n'a aucun moyen de coder les caractères â, é et à. Le standard ASCII est conçu pour l'anglais.

Character Sets

- ASCII est conçu pour l'anglais : impossible d'y coder â, é ou à, et encore moins le grec ou le thaï.
- ASCII n'utilise que **7 bits**, soit 128 caractères. Un octet en a **8** : 256 nombres, donc 128 de plus.
- Un **Character Set** garde la table ASCII pour les nombres de $(00)_{16}$ à $(7F)_{16}$, et utilise ceux de $(80)_{16}$ à $(FF)_{16}$ pour les caractères d'une langue ou d'un système d'écriture.
- À partir de 1987 sont créés des Character Sets : Latin 1, puis le grec, le cyrillique, l'arabe, l'hébreu, le thaï, etc.

$(00)_{16}$ à $(7F)_{16}$	$(80)_{16}$ à $(FF)_{16}$
ASCII identique dans tous les Character Sets	grec : α β Σ Ω
	thaï : ก ข ค
	etc.

Character Set : Latin 1

Le début est identique à la table ASCII.

Activité · 2 min

Quelles langues peut-on écrire avec ce codage ?

ISO/IEC 8859-1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	SHY	®	¯
B_	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C_	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D_	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E_	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F_	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Défauts des Character Sets

- Il faut se mettre d'accord en avance sur comment interpréter les nombres qu'on va échanger. Par exemple, comment sera compris le caractère grec Σ si on l'interprète dans le Character Set thaï ?
- Impossible de mélanger plusieurs langues dans un même document.
- 8 bits ne suffisent pas à représenter de nombreux systèmes d'écriture.

ISO/IEC 8859-7

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	‘	’	£	€	₯	¦	§	¨	©	ͺ	«	¬	SHY		—
B_	°	±	²	³	΄	΅	Ά	·	Έ	Ή	Ί	»	Ό	½	Ύ	Ώ
C_	ΐ	Α	Β	Γ	Δ	Ε	Ζ	Η	Θ	Ι	Κ	Λ	Μ	Ν	Ξ	Ο
D_	Π	Ρ		Σ	Τ	Υ	Φ	Χ	Ψ	Ω	Ϊ	Ϋ	ά	έ	ή	ί
E_	ΰ	α	β	γ	δ	ε	ζ	η	θ	ι	κ	λ	μ	ν	ξ	ο
F_	π	ρ	ς	σ	τ	υ	φ	χ	ψ	ω	ϊ	ϋ	ό	ύ	ώ	

ISO/IEC 8859-11

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	ก	ข	ฃ	ค	ฅ	ฆ	ง	จ	ฉ	ช	ซ	ฌ	ญ	ฎ	ฏ
B_	ฐ	ฑ	ฒ	ณ	ด	ต	ถ	ท	ธ	น	บ	ป	ผ	ฝ	พ	ฟ
C_	ภ	ม	ย	ร	ฤ	ล	ฦ	ว	ศ	ษ	ส	ห	ฬ	อ	ฮ	ฯ
D_	ะ	ั	า	ำ	ิ	ี	ึ	ื	ุ	ู	ฺ					฿
E_	เ	แ	โ	ใ	ไ	ๅ	ๆ	็	่	้	๊	๋	์	ํ	๎	๏
F_	๐	๑	๒	๓	๔	๕	๖	๗	๘	๙	๚	๛				

Unicode

Unicode est un *standard* informatique qui permet des échanges de textes dans différentes langues, à un niveau mondial. Il vise au codage de texte écrit en donnant à tout caractère de *n'importe quel système d'écriture* un nom et un *identifiant numérique*, et ce de manière unifiée, *quels que soient la plateforme informatique ou le logiciel utilisé*.

Le projet démarre en 1987 ; la première version est publiée en 1991.

Unicode aujourd'hui

Dans la 18ᵉ version du standard (qui date de 2026), on répertorie 172'808 caractères différents.

Nouveautés de la version 18 : blog.emojipedia.org/whats-new-in-unicode-18-0

بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

? ☃ 愛 が 🍣 𓀀

Unicode : attribuer des numéros aux caractères

Le standard Unicode se contente d'attribuer un nombre à chaque caractère.

Par exemple, la phrase :

L o l ␣ Γ ␣ 🐶

... se convertit en la suite de nombres suivante :

$(4C)_{16}$ $(6F)_{16}$ $(6C)_{16}$ $(20)_{16}$ $(393)_{16}$ $(20)_{16}$ $(1F436)_{16}$

Unicode : représentation sous forme d'octets

Par exemple, la phrase :

L o l ␣ Γ ␣ 🐶

… se convertit en la suite de nombres suivante :

$(4C)_{16}$ $(6F)_{16}$ $(6C)_{16}$ $(20)_{16}$ $(393)_{16}$ $(20)_{16}$ $(1F436)_{16}$

Le nombre de bits nécessaires pour représenter chacun de ces caractères est :

7 **7** **7** 6 **10** 6 **17**

Activité · 3 min

- Combien d'*octets* faut-il pour représenter l'espace ? le L ? le Γ ? le 🐶 ?
- Comment représenter cette séquence de nombres en *octets* ? Avez-vous des propositions, des idées ?

Nécessité d’un format de transformation

UTF-8 (1992)
Unicode **T**ransformation **F**ormat – **8** bits

- Utilisé par environ 99 % des sites Web (statistique d’octobre 2026).
- Permet de coder les caractères Unicode sous forme de **séquences de un à quatre octets**.
- *(Quelques alternatives sont UTF-16 et UTF-32.)*

UTF-8 : un codage à taille variable

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

- Le début (les bits à gauche) de chaque octet indique si le caractère est codé sur 1, 2, 3 ou 4 octets.
- Les octets qui complètent une séquence de 2, 3 ou 4 octets commencent par **`10`**.

Activité · 3 min

Combien de caractères de 2 octets différents peut-on représenter ?

Commencer par compter le nombre de bits utilisables. Donner votre réponse sous forme d'une puissance de 2.

Sommaire

1 **Codages du texte**
ASCII, Character Sets, Unicode avec UTF-8

2 **Exemples UTF-8**
Coder et décoder un caractère

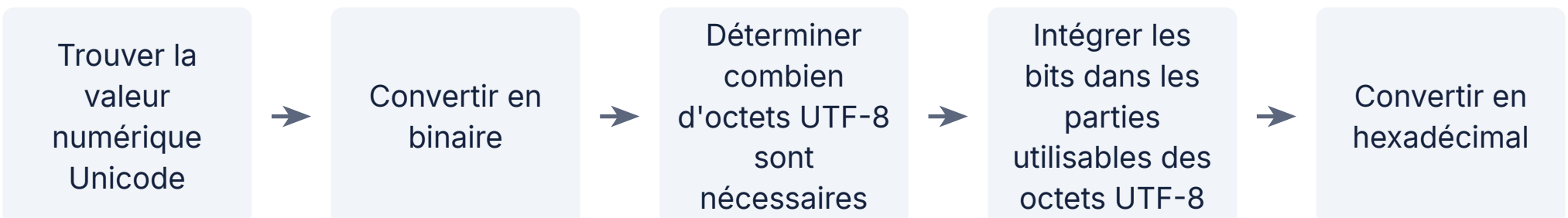
3 **Malentendus**
Sur les conventions

4 **Propriétés d'UTF-8**
Avantages et inconvénients

Coder la lettre é en UTF-8 : les phases

On cherche à coder le caractère **é** en UTF-8.

Il faut procéder en 5 phases :



Trouver la valeur numérique Unicode

Trouver la valeur numérique Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les parties utilisables des octets UTF-8 → Convertir en hexadécimal

On cherche à coder le caractère **é** en UTF-8.

On utilise ri.nwolff.info/unicode-converter pour trouver la correspondance :

é en hexadécimal : $(E9)_{16}$

é en décimal : $(233)_{10}$

Convertir en binaire

Trouver la valeur numérique Unicode → **Convertir en binaire** → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les parties utilisables des octets UTF-8 → Convertir en hexadécimal

Le caractère **é** a le code suivant en Unicode : $(E9)_{16}$

On convertit en binaire :

E	9
1110	1001

En Unicode, le caractère **é** a donc le code binaire : **11101001**

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Déterminer combien d'octets UTF-8 sont nécessaires

Trouver la valeur numérique Unicode → Convertir en binaire → **Déterminer combien d'octets UTF-8 sont nécessaires** → Intégrer les bits dans les parties utilisables des octets UTF-8 → Convertir en hexadécimal

En Unicode, le caractère **é** a le code binaire : **11101001**.
Ce nombre binaire contient **8 bits** d'information.

On cherche dans la table ci-dessous combien d'octets UTF-8 vont être nécessaires pour contenir ces 8 bits.
Ces 8 bits d'information doivent rentrer dans les « trous » des octets UTF-8.

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	`0.......`				7
2	`110.....`	`10......`			11
3	`1110....`	`10......`	`10......`		16
4	`11110...`	`10......`	`10......`	`10......`	21

- Sur 1 octet UTF-8, on ne peut faire entrer que 7 bits, parce que le premier bit est forcé à **0** : ça ne convient pas.
- Sur 2 octets UTF-8, on peut faire entrer 11 bits (5 dans le premier octet, 6 dans le deuxième), ce qui est *suffisant* pour représenter 8 bits. On va donc coder **é** sur 2 octets.

Intégrer les bits dans les parties utilisables des octets UTF-8

Trouver la valeur numérique Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → **Intégrer les bits dans les parties utilisables des octets UTF-8** → Convertir en hexadécimal

En Unicode, le caractère **é** a le code binaire : **11101001**, qui a une longueur de 8 bits.

8 bits entrent dans cette trame de 2 octets UTF-8 :	**110..... 10.....**
On commence à remplir les octets UTF-8 par la droite. Il y a la place pour 6 bits à droite. On découpe **11101001** en 2 groupes :	**11 101001**
On intègre ces bits dans les 2 octets UTF-8 :	**110...11 10101001**
On remplit les places vides avec des zéros :	**11000011 10101001**

Résultat : le nombre binaire **11101001** se code **11000011 10101001** en octets UTF-8.

Convertir en hexadécimal

Trouver la valeur numérique Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les parties utilisables des octets UTF-8 → **Convertir en hexadécimal**

On convertit les octets UTF-8 en hexadécimal :

```
1100 0011     1010 1001
  C    3        A    9
```

Résultat : **é** se code $(C3A9)_{16}$ en UTF-8.

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Coder é : résumé

1. En Unicode, le caractère **é** a le nombre hexadécimal $(\texttt{E9})_{16}$.
2. $(\texttt{E9})_{16}$ s'écrit `11101001` en binaire.
3. Ce nombre binaire fait 8 bits. 8 bits entrent dans 2 octets UTF-8 : `110.....` `10......`
4. On intègre `11101001` dans ces 2 octets : `11000011` `10101001`
5. `11000011` `10101001` s'écrit $(\texttt{C3A9})_{16}$ en hexadécimal.

Le caractère é se code $(\texttt{C3A9})_{16}$ en UTF-8.

UTF-8 : exercice de codage

En Unicode, le symbole Ω correspond au nombre hexadécimal $(3A9)_{16}$.

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Activité · 8 min

1. Combien de bits d'information permettent de représenter cette valeur ? (Il faut convertir depuis l'hexadécimal en utilisant la table à droite et compter les bits.)
2. Combien d'octets doit-on utiliser en UTF-8 pour coder ce nombre binaire ? Utiliser le tableau ci-dessous :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	`0.......`				7
2	`110.....`	`10......`			11
3	`1110....`	`10......`	`10......`		16
4	`11110...`	`10......`	`10......`	`10......`	21

3. Construire le codage UTF-8 binaire correspondant, à nouveau à l'aide du tableau ci-dessus.
4. Finalement, convertir ce nombre binaire en hexadécimal en utilisant la table à droite.

UTF-8 : exercice de codage

1. Combien de bits d'information permettent de représenter cette valeur ?

On convertit $(3A9)_{16}$ en binaire à l'aide de la table :

```
  3     A     9
0011  1010  1001
```

Sans les 0 de gauche, `11'1010'1001` : ce nombre binaire est fait de **10 bits**.

2. Combien d'octets doit-on utiliser en UTF-8 ?

Ces 10 bits sont codables sur **2 octets** :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	`0.......`				7
2	`110.....`	`10......`			11
3	`1110....`	`10......`	`10......`		16
4	`11110...`	`10......`	`10......`	`10......`	21

3. Construire le codage UTF-8 binaire correspondant.

On commence à remplir par la droite : 6 bits dans le 2e octet, 4 dans le premier.

```
Les bits découpés :          1110 101001
Intégrés dans les octets :   110.1110 10101001
Places vides remplies de 0 : 11001110 10101001
```

4. Convertir ce nombre binaire en hexadécimal.

```
1100 1110   1010 1001
   C    E      A    9
```

Ω se code $(CEA9)_{16}$ en UTF-8.

UTF-8 : décodage

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

Activité · 3 min

Vous recevez un texte complet de 10 octets de long codé en ASCII. Combien y a-t-il de caractères (lettres) dans ce texte ?

```
42 72 6F 75 69 6C 6C 61 72 64
```

Vous recevez un texte complet de 10 octets de long codé en UTF-8. Combien y a-t-il de caractères dans ce texte ?

```
C3 A9 74 C3 A9 20 F0 9F 98 8E
```

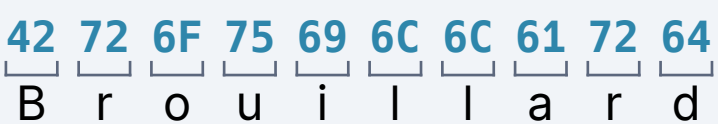
UTF-8 : décodage

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

Activité

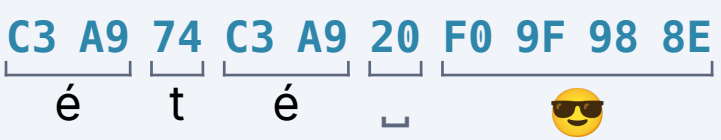
Vous recevez un texte complet de 10 octets de long codé en ASCII. Combien y a-t-il de caractères (lettres) dans ce texte ?

10 caractères.



Vous recevez un texte complet de 10 octets de long codé en UTF-8. Combien y a-t-il de caractères dans ce texte ?

5 caractères.



Un défaut d'UTF-8 (par rapport à ASCII, par exemple) est qu'il faut parcourir tout le texte pour savoir le nombre de caractères qu'il contient : il ne suffit pas de compter le nombre d'octets reçus.

Décoder : le premier caractère

On veut décoder la séquence UTF-8 d'octets :

C3 A9 74 C3 A9 20 F0 9F 98 8E

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0.......			
2	110.....	10......		
3	1110....	10......	10......	
4	11110...	10......	10......	10......

- On commence par le premier octet de la séquence (il est à gauche).
- On l'écrit en binaire : $(C3)_{16}$ = $(11000011)_2$.
- On identifie la forme à l'aide de la table UTF-8 : **110**00011.
- On en déduit le nombre d'octets utilisés pour le caractère. Ici, c'est 2 octets.
- On lit autant d'octets suivants qu'il faut. Ici, le caractère est sur 2 octets : on en a déjà lu un, il faut lire le deuxième et l'exprimer en binaire : $(A9)_{16}$ = $(10101001)_2$. Les deux premiers bits sont comme prévu **10**, comme tous les octets qui complètent un caractère.
- Pour ce caractère, l'information est codée dans les bits en couleur : **110**00011 **10**101001.
- On collecte ces bits : 00011101001.
- On supprime les 0 à gauche, qui ne servent à rien, et on arrange par groupes de 4 bits : 1110'1001.
- On convertit ce nombre binaire en hexadécimal, on obtient $(E9)_{16}$.
- On regarde dans la table Unicode (ou dans ri.nwolff.info/unicode-converter) pour connaître le caractère associé à ce nombre. En l'occurrence, il s'agit du caractère « **é** ».

Décoder : les caractères suivants

On veut décoder la séquence UTF-8 d'octets :

C3 A9 74 C3 A9 20 F0 9F 98 8E

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0.......			
2	110.....	10......		
3	1110....	10......	10......	
4	11110...	10......	10......	10......

- On continue avec $(74)_{16}$ = $(01110100)_2$.
- On identifie la forme à l'aide de la table UTF-8 : **01110100**.
- On en déduit le nombre d'octets utilisés pour le caractère. Ici, c'est 1 octet.
- Les caractères codés sur 1 octet sont très simples : on peut les décoder directement avec la table ASCII. Ici, on trouve le caractère « **t** ».

On a fini pour $(74)_{16}$.

- On reconnaît la séquence de 2 octets suivante, $(C3A9)_{16}$: c'est la même qu'au début, elle représente donc le caractère « **é** ».
- Le caractère suivant est $(20)_{16}$ = $(00100000)_2$: **00100000**. Il est donc codé sur 1 octet, on le trouve dans la table ASCII : c'est un espace.

Décoder : un caractère sur 4 octets

On veut décoder la séquence UTF-8 d'octets :

`C3 A9 74 C3 A9 20 F0 9F 98 8E`

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

On continue avec $(\texttt{F0})_{16}$ = $(\texttt{11110000})_2$: un caractère codé sur 4 octets. Il faut lire les 3 suivants, les convertir en binaire, et extraire les bits d'information :

Hexadécimal	F0	9F	98	8E
Binaire	11110000	10011111	10011000	10001110
Bits d'information	000	011111	011000	001110

On assemble les bits d'information qui viennent des 4 octets : `000 011111 011000 001110`

On les arrange par groupes de 4 bits sans copier les 0 de gauche, et on convertit en hexadécimal :

```
1 1111 0110 0000 1110
1   F    6    0    E
```

On cherche dans la table Unicode : $(\texttt{1F60E})_{16}$ représente le caractère 😎.

On est au bout de la séquence de 10 octets `C3 A9 74 C3 A9 20 F0 9F 98 8E`. Elle représente le texte de 5 caractères : « **été** 😎 ».

UTF-8 : exercice de décodage

Vous recevez le texte suivant codé en UTF-8 :

`50 69 C3 B1 61 74 61`

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

Activité · 6 min

1. À l'aide de ce tableau, déterminer combien de caractères sont écrits dans ce texte (indice : ce n'est pas obligatoirement 7, car certains caractères peuvent prendre plus d'un octet).
2. En utilisant cette table ASCII, décoder les caractères qui sont codés sur un octet.
3. Deviner (ou décoder) les caractères manquants à partir des caractères déjà trouvés.
4. Quel mot avez-vous reçu ?

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	*SP*	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

UTF-8 : exercice de décodage

1. Combien de caractères sont écrits dans ce texte ?

Tous les octets qui commencent par un symbole hexadécimal inférieur à 8 ont le bit de poids fort à **0** : d'après la table UTF-8, chacun de ces octets code un caractère : `50 69 C3 B1 61 74 61`

$(C3)_{16}$ commence par **110** et $(B1)_{16}$ par **10** : ils forment un seul caractère. Il y a donc **6 caractères**.

2. Décoder les caractères qui sont codés sur un octet.

À l'aide de la table ASCII, on arrive à décoder partiellement le message :

P i `C3 B1` a t a

3. Deviner (ou décoder) les caractères manquants.

Le caractère manquant commence par le symbole hexadécimal `C`, qui vaut `1100` en binaire : **110** est le marqueur d'un caractère UTF-8 sur 2 octets. On peut deviner qu'il s'agit du « **ñ** » (on pourrait aussi le décoder complètement si on avait une table Unicode).

4. Quel mot avez-vous reçu ?

`50`	`69`	`C3 B1`	`61`	`74`	`61`
P	i	ñ	a	t	a

À noter : le message a une longueur de 7 octets, mais ne contient que 6 caractères. C'est normal : UTF-8 est un codage à taille variable.

Sommaire

1 **Codages du texte**
ASCII, Character Sets, Unicode avec UTF-8

2 **Exemples UTF-8**
Coder et décoder un caractère

3 **Malentendus**
Sur les conventions

4 **Propriétés d'UTF-8**
Avantages et inconvénients

Un bug de codage



On voulait manifestement écrire un **é**.

En UTF-8, le codage de ce caractère est $(\texttt{C3A9})_{16}$.

Activité · 2 min

Pouvez-vous trouver dans ce cours un autre type de codage de texte qui pourrait expliquer ce qu'on voit à gauche ?

Un bug de codage



On voulait manifestement écrire un **é**.

En UTF-8, le codage de ce caractère est $(\texttt{C3A9})_{16}$.

Activité

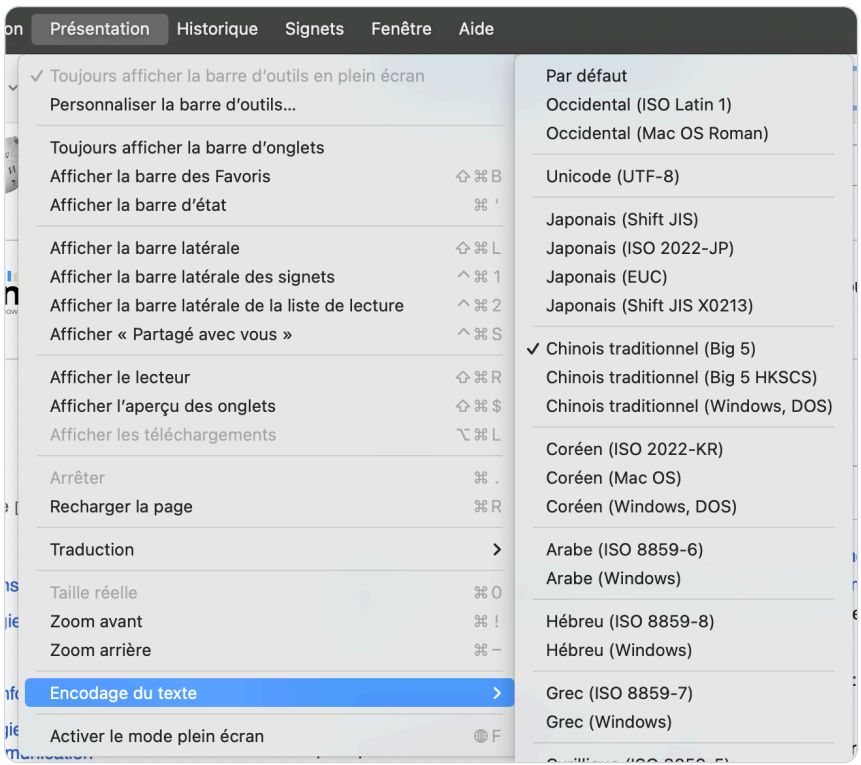
Pouvez-vous trouver dans ce cours un autre type de codage de texte qui pourrait expliquer ce qu'on voit à gauche ?

On a un problème de malentendu sur la convention utilisée pour représenter ce caractère :

- la personne qui a généré le message l'a fait en UTF-8 ;
- la personne qui a reçu le message l'a interprété comme Latin 1.

Démonstration dans Safari

1. Ouvrir la page fr.wikipedia.org/wiki/Informatique dans **Safari**.
2. Modifier l'encodage du texte (menu Présentation → Encodage du texte). Que voyez-vous ?



Sommaire

1. **Codages du texte**
   ASCII, Character Sets, Unicode avec UTF-8
2. **Exemples UTF-8**
   Coder et décoder un caractère
3. **Malentendus**
   Sur les conventions
4. **Propriétés d'UTF-8**
   Avantages et inconvénients

UTF-8 et ASCII

Les 128 premiers caractères d'Unicode sont les mêmes qu'en ASCII. Par exemple A → $(41)_{16}$, Z → $(5A)_{16}$.

Pour rappel, les règles UTF-8 pour représenter un caractère Unicode, selon le nombre de bits qui le composent :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

Activité · 2 min

- Comment est-ce qu'un texte ne contenant que des caractères ASCII, comme par exemple « Il doit y avoir une explication logique », va être encodé en UTF-8 ?
- Combien d'octets vont être utilisés pour représenter chaque caractère ?
- Est-ce que le résultat sera différent du texte codé en ASCII ?

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	*SP*	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

UTF-8 et ASCII

Les 128 premiers caractères d'Unicode sont les mêmes qu'en ASCII. Par exemple A → $(41)_{16}$, Z → $(5A)_{16}$.

Pour rappel, les règles UTF-8 pour représenter un caractère Unicode, selon le nombre de bits qui le composent :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0.......			
2	110.....	10......		
3	1110....	10......	10......	
4	11110...	10......	10......	10......

Activité

- Comment est-ce qu'un texte ne contenant que des caractères ASCII, comme par exemple « Il doit y avoir une explication logique », va être encodé en UTF-8 ?
- Combien d'octets vont être utilisés pour représenter chaque caractère ?
- Est-ce que le résultat sera différent du texte codé en ASCII ?

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	*SP*	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

UTF-8 est compatible avec ASCII.

UTF-8 : bits utiles et bits totaux

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables	Rapport bits utiles / bits totaux	Rapport en pourcentage
1	**0**`.......`				7	7 / 8	88 %
2	**110**`.....`	**10**`......`			11	11 / 16	69 %
3	**1110**`....`	**10**`......`	**10**`......`		16	16 / 24	67 %
4	**11110**`...`	**10**`......`	**10**`......`	**10**`......`	21	21 / 32	66 %

Activité · 2 min

Pourrait-on faire mieux ? C'est-à-dire améliorer le rapport bits utiles / bits totaux en changeant les valeurs du tableau ?

On pourrait être tenté de supprimer les marqueurs de suite de séquence d'octets (les **10** qui apparaissent 6 fois dans le tableau).

UTF-8 : un message pris en cours de route

Activité · 2 min

Vous recevez 50 octets de la suite d'un message – **dont vous avez raté le début** – qui commence par un octet de la forme « `10.....` ». Comment allez-vous faire pour interpréter la suite du texte ?

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

UTF-8 : un message pris en cours de route

Activité · 1 min

Vous recevez 50 octets de la suite d'un message – **dont vous avez raté le début** – qui commence par un octet de la forme « `10.....` ». Comment allez-vous faire pour interpréter la suite du texte ?

Il suffit de continuer à avancer dans le texte en sautant tous les octets qui commencent par `10`.

UTF-8 est auto-synchronisant.

Activité · 1 min

Combien d'octets allez-vous sauter au maximum ? Combien de caractères allez-vous sauter au maximum ?

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0.......`			
2	`110.....`	`10......`		
3	`1110....`	`10......`	`10......`	
4	`11110...`	`10......`	`10......`	`10......`

Hex Fiend : le fichier unicode-text

Activité · 7 min

Télécharger unicode-text.txt. Questions sur le fichier unicode-text :

- Comment sont encodées les lettres ?
- Comment est encodé le retour à la ligne ?
- Pourquoi est-ce qu'on ne voit pas les accents dans Hex Fiend ?
- Comment est encodé le é ? Le œ ? (Combien d'octets ?)
- Le caractère chinois (combien d'octets ?)
- Le hiéroglyphe (combien d'octets ?)
- On ajoute $(\texttt{C39F})_{16}$. Cette séquence UTF-8 représente quel caractère ?
- On ajoute $(\texttt{E298BA})_{16}$. Cette séquence UTF-8 représente quel caractère ?

Hex Fiend : le fichier glass.txt

Activité · 5 min

1. Télécharger glass.txt.
2. Arrivez-vous à voir le contenu dans TextEdit ?

Le fichier est corrompu : il a été obtenu en espionnant une conversation entre deux interlocuteurs, mais en commençant au milieu de la conversation (il nous manque les premiers octets du message).

On sait que c’est un fichier texte codé en Unicode avec UTF-8.

3. Examiner le fichier dans Hex Fiend, examiner le début, trouver une manière de réparer le fichier pour qu’on puisse lire la suite du message.

Sources

Contenu et outils

Wikipédia – Code Morse international
fr.wikipedia.org

Wikipédia – Caractère (typographie)
fr.wikipedia.org

Wikipédia – American Standard Code for Information Interchange
fr.wikipedia.org

ISO – la norme ISO 8859-1:1987, Latin 1
iso.org

Convertisseur Unicode
ri.nwolff.info

ridiculousfish – Hex Fiend
hexfiend.com

Unicode et UTF-8

Wikipédia – Unicode
fr.wikipedia.org

Unicode Consortium – l'histoire d'Unicode
unicode.org

Unicode Consortium – Unicode 1.0
unicode.org

Unicode Consortium – Unicode 18.0
unicode.org

Compart – le caractère U+00E9, é
compart.com

Wikipédia – UTF-8
fr.wikipedia.org

W3Techs – l'usage d'UTF-8 sur le Web
w3techs.com

Images

Hans Hillewaert – la pierre de Rosette (CC BY-SA 4.0)
commons.wikimedia.org

Détournement anonyme d'une couverture de *Martine* (Marcel Marlier, Casterman)